

Trabalhando com Arquivos em Java

Parte 3 — *RandomAccessFile e Serialization*

Acesso binário e posicional a arquivos com `RandomAccessFile` e índices, leitura e escrita de tipos primitivos com `DataOutputStream/DataInputStream`, e o mecanismo de serialização de objetos do Java. Exemplos aplicados a um sistema de folha de pagamento com registros de funcionários.

CONTEÚDO DA APOSTILA

- 01 `RandomAccessFile` e o Ponteiro de Arquivo
- 02 Índices: Localizando Registros Rapidamente
- 03 `DataOutputStream` e `DataInputStream`
- 04 Serialization: Object Streams
- 05 `serialVersionUID` e Mudanças Compatíveis
- 06 O Modificador `transient`

Sumário

0 1 **RandomAccessFile e o Ponteiro de Arquivo**

Acessando e modificando dados em qualquer posição de um arquivo

0 2 **Índices: Localizando Registros Rapidamente**

Como montar um índice para não precisar ler o arquivo inteiro

0 3 **DataOutputStream e DataInputStream**

Gravando e lendo tipos primitivos Java de forma portátil

0 4 **Serialization: Object Streams**

Transformando objetos inteiros em bytes, e de volta

0 5 **serialVersionUID e Mudanças Compatíveis**

Como evoluir uma classe serializável sem quebrar dados já gravados

0 6 **O Modificador transient**

Excluindo campos específicos do processo de serialização

RandomAccessFile e o Ponteiro de Arquivo

Acessando e modificando dados em qualquer posição de um arquivo

Até aqui, os métodos de leitura e escrita apresentados percorrem o arquivo de forma sequencial, do início ao fim. A classe **RandomAccessFile** oferece uma alternativa: a capacidade de acessar e modificar dados diretamente em qualquer posição específica dentro do arquivo. Um **RandomAccessFile** se comporta como um grande array de bytes armazenado no sistema de arquivos, e essa classe pode ser usada tanto para leitura quanto para escrita.

Existe uma espécie de cursor, ou índice, apontando para esse array implícito: o **file pointer** (ponteiro de arquivo). O **RandomAccessFile** lê e escreve dados binários usando métodos especiais, que já sabem controlar quantos bytes serão lidos ou escritos em cada operação.

Movendo o ponteiro de arquivo

Ao abrir um **RandomAccessFile**, seu ponteiro começa na posição 0, o início do arquivo. Para mover o ponteiro, chama-se o método **seek**, passando um valor **long** com a posição desejada. Para consultar a posição atual, usa-se **getFilePointer**. Dependendo do método de leitura ou escrita usado, o ponteiro avança automaticamente um número específico de bytes ao final da operação.

```
// Abrindo o arquivo de folha de pagamento para leitura e escrita
try (RandomAccessFile raf = new RandomAccessFile("folha.dat", "rw")) {
    System.out.println(raf.getFilePointer()); // 0
    raf.seek(120); // move o ponteiro para a posição 120
    int idFuncionario = raf.readInt();
    double salario = raf.readDouble();
    raf.seek(124); // volta e sobrescreve apenas o salário
    raf.writeDouble(salario * 1.10);
} catch (IOException e) {
    System.out.println("Erro ao acessar folha de pagamento: " + e.getMessage());
}
```

Por que usar RandomAccessFile?

Imagine um arquivo com muitos milhões de registros, e que, em um dado momento, você só precisa acessar cerca de 50 deles. Em vez de carregar um milhão de registros para a memória, é possível manter apenas um pequeno array ou mapa que indique como localizar os registros de interesse dentro do arquivo grande. Você certamente não gostaria de começar a leitura do início do arquivo e percorrer dez milhões de registros verificando um a um até achar o certo. O **RandomAccessFile** permite avançar ou retroceder diretamente até uma posição no arquivo usando o método **seek** — e a partir dali, ler apenas os dados que realmente importam para a aplicação.

NOTA

Para usar essa estratégia com eficiência, é preciso saber de antemão quantos registros existem no arquivo, qual o tamanho de cada registro, e como cada registro será identificado para ser recuperado depois.

Índices: Localizando Registros Rapidamente

Registros de tamanho fixo, variável, e onde guardar o índice

Registros de tamanho fixo

Um `RandomAccessFile` precisa de um índice, que guarda a posição do ponteiro de arquivo associada a cada registro de interesse. Esse índice pode ser **implícito** quando o arquivo tem registros de largura fixa e você só precisa localizar dados por número de linha — nesse caso, basta uma multiplicação simples. Para obter o décimo milésimo registro, quando todos os registros têm 250 caracteres de comprimento, basta calcular 10.000×250 para apontar diretamente para a posição do registro desejado.

É mais comum, no entanto, precisar recuperar registros por um identificador não sequencial, em vez de um número de linha. Para isso, é necessário um índice de verdade, contendo esse identificador e a posição no arquivo do registro associado a ele. Para registros de largura fixa, esse índice nem precisa guardar a posição do ponteiro — basta associar o número da linha ao identificador do registro.

Índice da linha	ID do Registro	Posição no Arquivo	Registro de tamanho fixo (250 chars)
0	100000	0	Primeiro registro
1	1	250	Segundo registro
2	543210	500	Terceiro registro
3	777	750	Quarto registro

Registros de tamanho variável

Para registros de tamanho variável, o número da linha sozinho não é informação suficiente para calcular o ponteiro de arquivo. É possível guardar o tamanho de cada registro no índice, mas o mais comum é simplesmente guardar a posição inicial de cada registro.

Índice	ID do Registro	Ponteiro de Arquivo	Tamanho Variável
0	100000	0	Primeiro registro (50)
1	1	50	Segundo registro (250)
2	543210	300	Terceiro registro (150)
3	777	450	Quarto registro (500)

Onde fica o índice?

Em um arquivo de largura fixa, o índice pode nem existir fisicamente. Quando existe, costuma estar em uma de três posições: no início do arquivo de dados, antes dos registros; no final do arquivo de dados, depois de todos os registros; ou em um arquivo totalmente separado.

Uma estratégia comum é escrever primeiro os registros a partir de uma posição conhecida do arquivo (reservando espaço para o índice no início), guardando a posição de cada um em um mapa em memória à medida que são gravados. Depois de inserir todos os registros, grava-se a contagem total na posição 0 do arquivo (como um inteiro, 4 bytes), seguida por cada par chave-valor do índice — cada entrada indexada ocupando 16 bytes (um int de 4 bytes para o ID mais um long de 8 bytes para a posição, com folga). Como se trata de um `RandomAccessFile`, é perfeitamente possível começar escrevendo os registros antes de o índice existir, bastando calcular com antecedência a posição inicial: $(quantidadeDeRegistros \times 16) + 4$.

```
// Gravando registros de funcionários e montando o índice em memória
Map<Integer, Long> indice = new LinkedHashMap<>();
int totalFuncionarios = funcionarios.size();
long posicaoInicial = (totalFuncionarios * 16L) + 4;
try (RandomAccessFile raf = new RandomAccessFile("folha.dat", "rw")) {
    long posicaoAtual = posicaoInicial;
    raf.seek(posicaoAtual);
    for (Funcionario f : funcionarios) {
        indice.put(f.getId(), raf.getFilePointer());
        raf.writeInt(f.getId());
        raf.writeDouble(f.getSalario());
        raf.writeUTF(f.getNome());
    }
    raf.seek(0);
    raf.writeInt(totalFuncionarios);
    for (Map.Entry<Integer, Long> entrada : indice.entrySet()) {
        raf.writeInt(entrada.getKey());
        raf.writeLong(entrada.getValue());
    }
}
```

DataOutputStream e DataInputStream

Gravando e lendo tipos primitivos Java de forma portátil

Um **DataOutputStream** permite que uma aplicação escreva tipos primitivos Java em um stream de saída, de forma portátil — ou seja, de um jeito que qualquer outra aplicação Java consiga interpretar corretamente, independentemente da plataforma. Do outro lado, uma aplicação pode usar um **DataInputStream** para ler esses mesmos dados de volta, na mesma ordem em que foram escritos.

```
// Gravando e lendo um registro de funcionário com Data streams
Path arquivo = Paths.get("folha", "funcionario-42.dat");
try (DataOutputStream saida = new DataOutputStream(
    new BufferedOutputStream(Files.newOutputStream(arquivo)))) {
    saida.writeInt(42);
    saida.writeUTF("Marina Alcântara");
    saida.writeDouble(7850.00);
}
try (DataInputStream entrada = new DataInputStream(
    new BufferedInputStream(Files.newInputStream(arquivo)))) {
    int id = entrada.readInt();
    String nome = entrada.readUTF();
    double salario = entrada.readDouble();
    System.out.printf("#%d %s - R$ %.2f%n", id, nome, salario);
}
```

ATENÇÃO

A ordem de leitura precisa espelhar exatamente a ordem de escrita — Data streams não guardam nenhum tipo de metadado sobre os campos, apenas os bytes brutos, na sequência em que foram gravados.

Serialization: Object Streams

Transformando objetos inteiros em bytes, e de volta

Serialização é o processo de traduzir uma estrutura de dados ou um objeto inteiro para um formato que pode ser armazenado em arquivo (ou transmitido pela rede). Apenas instâncias de classes **Serializable** podem ser serializadas — ou seja, a classe precisa implementar a interface **Serializable**. Essa interface não declara nenhum método; ela serve apenas como marcação, informando ao Java que aquela classe pode ser serializada. Todos os subtipos de uma classe serializável também são, por consequência, serializáveis.

O mecanismo padrão de serialização grava a classe do objeto, a assinatura da classe, e os valores de todos os campos não estáticos. Essas informações são usadas para restaurar o objeto e seu estado durante a leitura — um processo chamado **desserialização**, ou reconstituição do objeto.

```
// Serializando e desserializando um registro de funcionário
public class Funcionario implements Serializable {
    private int id;
    private String nome;
    private double salario;
    // construtor, getters e setters omitidos
}

// Escrevendo o objeto inteiro em um arquivo:
Funcionario f = new Funcionario(42, "Marina Alcântara", 7850.00);
try (ObjectOutputStream saida = new ObjectOutputStream(
    Files.newOutputStream(Paths.get("folha", "funcionario-42.ser")))) {
    saida.writeObject(f);
}

// Reconstituindo o objeto a partir do arquivo:
try (ObjectInputStream entrada = new ObjectInputStream(
    Files.newInputStream(Paths.get("folha", "funcionario-42.ser")))) {
    Funcionario reconstituído = (Funcionario) entrada.readObject();
    System.out.println(reconstituído.getNome());
} catch (ClassNotFoundException e) {
    System.out.println("Classe não encontrada ao desserializar.");
}
```

NOTA

`readObject` declara `ClassNotFoundException` além de `IOException`, porque a JVM que está lendo o objeto precisa localizar a classe correspondente no classpath para reconstituí-lo — e isso pode falhar se a classe não estiver disponível.

serialVersionUID e Mudanças Compatíveis

Como evoluir uma classe serializável sem quebrar dados já gravados

Implementar `Serializable` e usar `writeObject/readObject` com `Object streams` parece resolver tudo — mas essa é só parte da história. Toda classe serializável tem um campo chamado **serialVersionUID**, criado implicitamente pelo compilador quando não é declarado explicitamente. Esse valor é calculado com base em detalhes da classe, como o número de campos, seus tipos e suas declarações. Ao ler um objeto de um stream, o runtime confere o `serialVersionUID` armazenado contra o da classe atual — se eles não coincidirem, há um problema de compatibilidade, e o runtime lança uma `InvalidClassException`.

Como diferentes compiladores podem gerar esse campo implícito de forma diferente, é fortemente recomendado declará-lo explicitamente como um campo privado e estático.

```
// Declarando o serialVersionUID explicitamente
public class Funcionario implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String nome;
    private double salario;
}
```

O que é uma mudança incompatível

Tipo de mudança	Exemplos
Incompatível	Alterar o tipo declarado de um campo primitivo · Remover campos · Transformar um campo não estático em estático, ou um campo não <code>transient</code> em <code>transient</code>
Compatível	Adicionar novos campos · Adicionar métodos <code>writeObject</code> e <code>readObject</code> · Alterar o modificador de acesso de um campo · Transformar um campo estático em não estático, ou em <code>transient</code>

Vale notar que o processo de serialização de um campo do tipo objeto inclui informações sobre o tipo do objeto e seu supertipo. Por isso, trocar, por exemplo, um campo `ArrayList` por um `LinkedList` costuma desserializar sem problemas — as duas classes compartilham um supertipo em comum (`List`).

DICA

Antes de fazer qualquer alteração em uma classe já usada para serializar dados existentes, vale consultar a especificação oficial de mudanças compatíveis e incompatíveis do Java, disponível na documentação da Oracle.

O Modificador transient

Excluindo campos específicos do processo de serialização

O modificador **transient** indica que um campo não deve ser serializado. Isso é útil para variáveis que contêm informações sensíveis — como uma senha ou um token de autenticação — ou simplesmente para campos que não precisam ser persistidos por outros motivos, como um valor calculado em tempo de execução ou uma referência a um recurso que não faz sentido fora daquela sessão da aplicação.

```
// Excluindo dados sensíveis da serialização de um funcionário
public class Funcionario implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String nome;
    private double salario;
    private transient String senhaTemporaria; // nunca vai para o arquivo
    // getters, setters e construtor omitidos
}
```

Ao desserializar um objeto, qualquer campo marcado como transient volta com o valor padrão do seu tipo — null para objetos, 0 para tipos numéricos, false para boolean — já que ele nunca chegou a ser gravado no stream.

ATENÇÃO

Marcar um campo como transient não o protege de outras formas de acesso, como reflection — ele apenas garante que o mecanismo padrão de serialização o ignore. Para dados verdadeiramente sensíveis, combine transient com outras práticas de segurança, como nunca manter a informação em memória por mais tempo do que o necessário.

Fim de apostila

Esta apostila completou o módulo de Input/Output em Java: acesso posicional com RandomAccessFile, índices para localizar registros com eficiência, tipos primitivos com Data streams, e o mecanismo de serialização de objetos — incluindo como evoluir uma classe serializável com segurança. Com as três apostilas desta série, você tem uma visão completa de como o Java se comunica com o sistema de arquivos.

JavaStudiesHub